

INT234: PREDICTIVE ANALYTICS

ANALYSIS AND COST PREDICTION OF ELECTRIC VEHICLE CHARGING PATTERNS

(Project Semester August–December 2026)

Submitted by

Kunwar Prakhar Singh

Registration No: 12502064

Section: K2P24QR

Course Code: INT234

Under the Guidance of

Dr. Mrinalini Rana

CERTIFICATE

This is to certify that **Kunwar Prakhar Singh** bearing Registration no**. 12502064** has completed INT234 project titled, "**Analysis and Cost Prediction of Electric Vehicle Charging Patterns**" under my guidance and supervision. To the best of my knowledge, the present work is the result of his original development, effort and study.

Name of the Supervisor: **Dr. Mrinalini Rana**
Lovely Professional University
Phagwara, Punjab.

DECLARATION

I, Kunwar Prakhar Singh, student of Bachler of Technology under CSE Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Registration No. 12502064
Kunwar Prakhar Singh

Date: 28/10/2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty guide for the guidance, encouragement and helpful feedback given throughout this project. Their explanations of data analysis and machine learning concepts made it possible for me to complete this work.

I am thankful to Lovely Professional University and the Discipline of CSE/IT for providing the opportunity, the learning environment and the resources needed to carry out this project.

I also thank the creators of the open "**Electric Vehicle Charging Patterns**" dataset on Kaggle, and the developers of the free Python libraries Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn, which were used in this project.

Finally, I thank my family and friends for their constant support and motivation.

Kunwar Prakhar Singh

TABLE OF CONTENTS

1. Introduction **7**
1.1 Background 7
1.2 Problem Statement 7
1.3 Objectives of the Project 7
1.4 Tools and Technologies Used 8
1.5 Methodology 8
1.6 Organisation of the Report 9
2. Source of Dataset **10**
2.1 Dataset Details 10
2.2 Description of Columns 10
2.3 Sample of the Data 11
2.4 Why this Dataset was Chosen 11
2.5 Project Repository 12
3. EDA Process **13**
3.1 Loading the Data 13
3.2 Understanding the Structure 13
3.3 Handling Missing Values 13
3.4 Checking Duplicates 14
3.5 Checking for Impossible Values 14
3.6 Dropping Unneeded Columns 14
3.7 Feature Engineering 14
3.8 Summary Statistics 15
3.9 Distribution of Numeric Columns 16
3.10 Key Observations from EDA 16
4. Analysis on Dataset **17**
4.1 Analysis 1: User and Vehicle Profile 17
4.2 Analysis 2: Charging Cost and Energy by Location and Charger Type 19
4.3 Analysis 3: Time-based Charging Patterns 22
4.4 Analysis 4: Relationships between Numeric Factors 24
4.5 Analysis 5: Predicting Charging Cost with Machine Learning 27
5. Conclusion **32**
6. Future Scope **33**
7. References **34**

LIST OF FIGURES

Figure 1.1: Project workflow 8
Figure 3.1: Histograms of key numeric columns 16
Figure 4.1: Number of sessions by vehicle model, user type and charger type 18
Figure 4.2: Average charging cost by city 20
Figure 4.3: Average charging cost by charger type 21
Figure 4.4: Distribution of charging cost (red line = mean) 21
Figure 4.5: Number of sessions by time of day and by day of week 23
Figure 4.6: Average charging cost on weekdays vs weekends 24
Figure 4.7: Correlation heatmap of numeric columns 26
Figure 4.8: Scatter plots: energy vs cost, and duration vs energy 26
Figure 4.9: Model comparison by Mean Absolute Error (lower is better) 30
Figure 4.10: Model comparison by R^2 score (higher is better) 30
Figure 4.11: Random Forest: actual vs predicted charging cost 31
Figure 4.12: Top 10 feature importances from the Random Forest 31

LIST OF TABLES

Table 1.1: Tools and libraries used 8
Table 2.1: Dataset overview 10
Table 2.2: Description of the 20 columns 10
Table 2.3: First five rows (selected columns) 11
Table 3.1: Data quality checks and their results 14
Table 3.2: Summary of data preparation steps 15
Table 3.3: Summary statistics of numeric columns 15
Table 4.1: Session counts by vehicle model, user type and charger type 17
Table 4.2: Crosstab of user type and charger type 18
Table 4.3: Average cost and energy by city 19
Table 4.4: Average cost, energy and duration by charger type 19
Table 4.5: Sessions and average cost by time of day 22
Table 4.6: Sessions by day of week 22
Table 4.7: Weekday vs weekend comparison 23
Table 4.8: Correlation of each numeric column with charging cost 25
Table 4.9: The four regression models used 27
Table 4.10: Evaluation metrics 28
Table 4.11: Performance of the four models and the baseline on test data 29

1. Introduction

1.1 Background

Electric Vehicles (EVs) are becoming common around the world because they produce no exhaust emissions and are cheaper to run than petrol or diesel cars. As more people buy EVs, the demand for public charging stations also grows. Charging station operators, city planners and electricity companies need to understand how people charge their vehicles: which chargers they use, at what time they charge, how much energy they take and how much they pay.

Data Science helps answer these questions. By collecting data about charging sessions and analysing it with simple tools, we can find patterns and test whether the cost of a charging session can be predicted in advance. This project applies basic Data Science and Machine Learning techniques to a dataset of 1,320 EV charging sessions.

1.2 Problem Statement

The aim of this project is to explore an EV charging sessions dataset to understand user behaviour, charging cost, time patterns and relationships between variables, and then to check whether simple machine learning models can predict the **charging cost (USD)** of a session from the other recorded details.

1.3 Objectives of the Project

The project has five analytical objectives:

1. **User and vehicle profile:** Find which vehicle models, user types and charger types are most common.
2. **Cost and energy by location and charger type:** Compare the average charging cost and energy used across cities and charger types.
3. **Time-based charging patterns:** Study when people charge (time of day, day of week) and compare weekend and weekday sessions.
4. **Relationships between numeric factors:** Measure how strongly numeric columns such as energy, duration, cost and temperature are related, using correlation.

5. **Predicting charging cost:** Train and compare four simple regression models (Linear Regression, Decision Tree, K-Nearest Neighbors and Random Forest) against a simple baseline.

1.4 Tools and Technologies Used

Table 1.1: Tools and libraries used

Tool / Library	Purpose in this project
Python 3	Programming language used for the whole project
Jupyter Notebook	Running code step by step and showing outputs and charts
Pandas	Loading the CSV file, cleaning data, grouping and summarising
NumPy	Basic numeric operations (for example square root for RMSE)
Matplotlib	Bar charts, histograms and scatter plots
Seaborn	Correlation heatmap and chart styling
Scikit-learn	Train/test split, scaling, the four ML models and evaluation metrics

1.5 Methodology

The project follows a simple step-by-step workflow, shown in Figure 1.1. First the dataset is loaded and understood. Then it is checked and prepared (missing values, duplicates and impossible values checked, unneeded columns dropped) and one new feature is created. After that, exploratory analysis is done for the first four objectives using tables and charts. Finally, four machine learning models are trained on 80% of the data and tested on the remaining 20%.

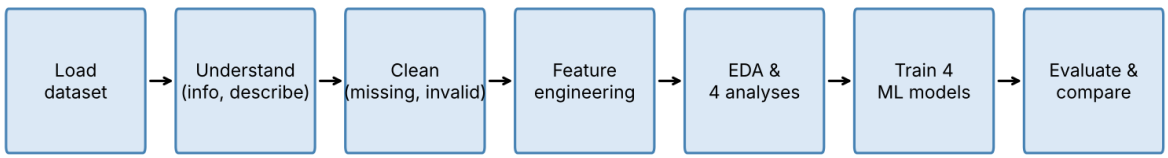


Figure 1.1: Project workflow

1.6 Organisation of the Report

Chapter 2 describes the source and structure of the dataset. Chapter 3 explains the Exploratory Data Analysis (EDA) process, including data cleaning. Chapter 4 presents the five analyses, each with an introduction, a general description, the functions and formulas used, the results and the visualizations. Chapter 5 gives the conclusion, Chapter 6 the future scope and Chapter 7 the references.

2. Source of Dataset

2.1 Dataset Details

The dataset used in this project is the **Electric Vehicle Charging Patterns** dataset, published publicly on Kaggle by the user *valakhorasani* in October 2024. It is a single CSV file named ev_charging_patterns.csv. Each row represents one charging session, with details about the vehicle, the user, the charging station, the time of charging, the energy used and the cost.

Table 2.1: Dataset overview

Property	Value
Dataset name	Electric Vehicle Charging Patterns
Source	Kaggle (kaggle.com/datasets/valakhorasani/electric-vehicle-charging-patterns)
File name / format	ev_charging_patterns.csv (comma-separated values)
Number of rows	1,320 charging sessions
Number of columns	20
Numeric columns	10
Text (categorical) columns	10
Time period of sessions	01 Jan 2024 to 24 Feb 2024
Cities covered	Chicago, Houston, Los Angeles, New York, San Francisco

2.2 Description of Columns

Table 2.2: Description of the 20 columns

No.	Column name	Type	Description
1	User ID	Text	Unique ID of the user (different for every row)
2	Vehicle Model	Text	EV model, e.g. Tesla Model 3, Nissan Leaf
3	Battery Capacity (kWh)	Numeric	Total battery size of the vehicle
4	Charging Station ID	Text	ID of the charging station used
5	Charging Station Location	Text	City of the station
6	Charging Start Time	Text	Date and time when charging started
7	Charging End Time	Text	Date and time when charging ended
8	Energy Consumed (kWh)	Numeric	Energy delivered to the battery in the session
9	Charging Duration (hours)	Numeric	How long the session lasted
10	Charging Rate (kW)	Numeric	Average charging power
11	Charging Cost (USD)	Numeric	Amount paid for the session (target for prediction)
12	Time of Day	Text	Morning, Afternoon, Evening or Night
13	Day of Week	Text	Monday to Sunday
14	State of Charge (Start %)	Numeric	Battery % at the start of charging
15	State of Charge (End %)	Numeric	Battery % at the end of charging
16	Distance Driven (since last charge) (km)	Numeric	Distance driven since the previous charge

17	Temperature (°C)	Numeric	Outside temperature during charging
18	Vehicle Age (years)	Numeric	Age of the vehicle
19	Charger Type	Text	Level 1, Level 2 or DC Fast Charger
20	User Type	Text	Commuter, Casual Driver or Long-Distance Traveler

2.3 Sample of the Data

Table 2.3 shows the first five rows of a few important columns, so the reader can see what the raw data looks like.

Table 2.3: First five rows (selected columns)

Vehicle	City	Energy (kWh)	Duration (h)	Cost (USD)	Charger
BMW i3	Houston	60.71	0.59	13.09	DC Fast Charger
Hyundai Kona	San Francisco	12.34	3.13	21.13	Level 1
Chevy Bolt	San Francisco	19.13	2.45	35.67	Level 2
Hyundai Kona	Houston	79.46	1.27	13.04	Level 1
Hyundai Kona	Los Angeles	19.63	2.02	10.16	Level 1

2.4 Why this Dataset was Chosen

- It is about a modern and important topic (electric mobility).
- It is small enough (1,320 rows) to run quickly on any laptop, which makes it suitable for learning.
- It contains both numeric columns (energy, cost, duration) and categorical columns (city, charger type, user type), so it allows many kinds of analysis.
- It has a clear numeric target (Charging Cost) that can be used for a regression task.
- It allows practice of standard data-quality checks (missing values, duplicates and impossible percentages).

2.5 Project Repository

All the project files, including the Jupyter notebook (finalp.ipynb), are available in the GitHub repository:

https://github.com/KunwarPrakharSingh12/INT234_CA2

3. EDA Process

Exploratory Data Analysis (EDA) means looking at the data carefully before building any model. The purpose is to understand what each column contains, find errors, fix them, and get a first idea of patterns. The steps followed are described below.

3.1 Loading the Data

The CSV file is loaded into a Pandas DataFrame, which is a table made of rows and columns.

```
import pandas as pd
df = pd.read_csv("ev_charging_patterns.csv")
print(df.shape)         # (1320, 20)
```

3.2 Understanding the Structure

The function df.info() lists every column with its data type and the number of filled (non-null) values. It showed that 10 columns are numeric (float64) and 10 are text, and that every column has 1,320 filled values. The function df.describe() gives summary statistics such as mean, minimum and maximum for each numeric column.

3.3 Handling Missing Values

The function df.isnull().sum() counts the empty cells in each column. In the copy of the file used in this project, **no column has missing values**.

For comparison, the original file on Kaggle has 66 missing values in each of three columns (Energy Consumed (kWh), Charging Rate (kW), Distance Driven (since last charge) (km)) and 32 rows with a battery percentage above 100%. The copy used here appears to have been pre-cleaned: the gaps are already filled with each column's median and the battery percentages stop at 100%.

The notebook still contains the step that fills missing values with the **median**, as a safety check. The median is the middle value when all values are sorted. It is preferred over the mean because a few very large values (outliers) can pull the mean up, but they do not affect the median much. In this file the step changes nothing ("Missing values left: 0").

```
print(df.isnull().sum()[df.isnull().sum() > 0])     # empty: no missing
values

for col in ["Energy Consumed (kWh)", "Charging Rate (kW)",
            "Distance Driven (since last charge) (km)"]:
```

```
    df[col] = df[col].fillna(df[col].median())
```

Table 3.1: Data quality checks and their results

Check	Function used	Result in this file
Missing values	df.isnull().sum()	0 in every column
Duplicate rows	df.duplicated().sum()	0
Battery % above 100	boolean filter on State of Charge	0 rows (maximum is 100%)
Unneeded columns	df.drop(columns=[...])	4 columns dropped

3.4 Checking Duplicates

The function df.duplicated().sum() returned **0**, so there are no repeated rows in the dataset.

3.5 Checking for Impossible Values

State of Charge is a battery percentage, so it must be between 0 and 100. The notebook keeps only rows where both the start and end percentages are 100 or less. In this file the maximum is exactly 100%, so the filter removed **0 rows** and all **1,320 rows** remain for analysis. The check is still useful: if the code is run on the original Kaggle file, it would catch the 32 rows with values above 100%.

```
rows_before = len(df)
df = df[(df["State of Charge (Start %)"] <= 100) &
        (df["State of Charge (End %)"] <= 100)]
print("Rows removed:", rows_before - len(df))   # 0
```

3.6 Dropping Unneeded Columns

- **User ID** – every row has a different value, so it carries no pattern.
- **Charging Station ID** – 462 different stations with only a few sessions each; too many categories to be useful.
- **Charging Start Time / End Time** – the useful time information is already available in *Time of Day*, *Day of Week* and *Charging Duration*.

3.7 Feature Engineering

Feature engineering means creating a new useful column from existing ones. One simple feature was created: **Is Weekend**, which is 1 if the session happened on a Saturday or Sunday and 0 otherwise. After this step, 924 sessions are on weekdays and 396 on weekends. This column is used in Objective 3 and as an input for the models.

```
df["Is Weekend"] = df["Day of Week"].isin(["Saturday",
"Sunday"]).astype(int)
```

Table 3.2: Summary of data preparation steps

Step	Rows	Columns
Dataset as loaded	1,320	20
After missing-value check	1,320	20
After battery % check	1,320	20
After dropping 4 columns	1,320	16
After adding Is Weekend	1,320	17

3.8 Summary Statistics

Table 3.3: Summary statistics of numeric columns

Column	Mean	Std. dev.	Min	Median	Max
Battery Capacity (kWh)	74.53	20.63	1.53	75.00	193.00
Energy Consumed (kWh)	42.65	21.84	0.05	42.69	152.24
Charging Duration (hours)	2.27	1.06	0.10	2.26	7.64
Charging Rate (kW)	25.95	13.66	1.47	25.60	97.34
Charging Cost (USD)	22.55	10.75	0.23	22.08	69.41
State of Charge (Start %)	49.00	23.73	2.33	48.24	100.00
State of Charge (End %)	74.70	15.82	7.60	75.68	100.00
Distance Driven (since last charge) (km)	153.53	83.83	0.86	152.26	398.36
Temperature (°C)	15.26	14.83	-10.72	14.63	73.17
Vehicle Age (years)	3.61	2.31	0.00	4.00	11.69

The average session uses about 42.6 kWh of energy, lasts about 2.27 hours and costs about USD 22.55. Temperature ranges from -10.7 °C to 73.2 °C; the maximum is unusually high and is likely an outlier, but it was kept because it is not physically impossible in the way a battery above 100% is.

3.9 Distribution of Numeric Columns

A histogram divides the range of a column into small intervals (bins) and shows how many rows fall into each one. Figure 3.1 shows histograms for six key columns.

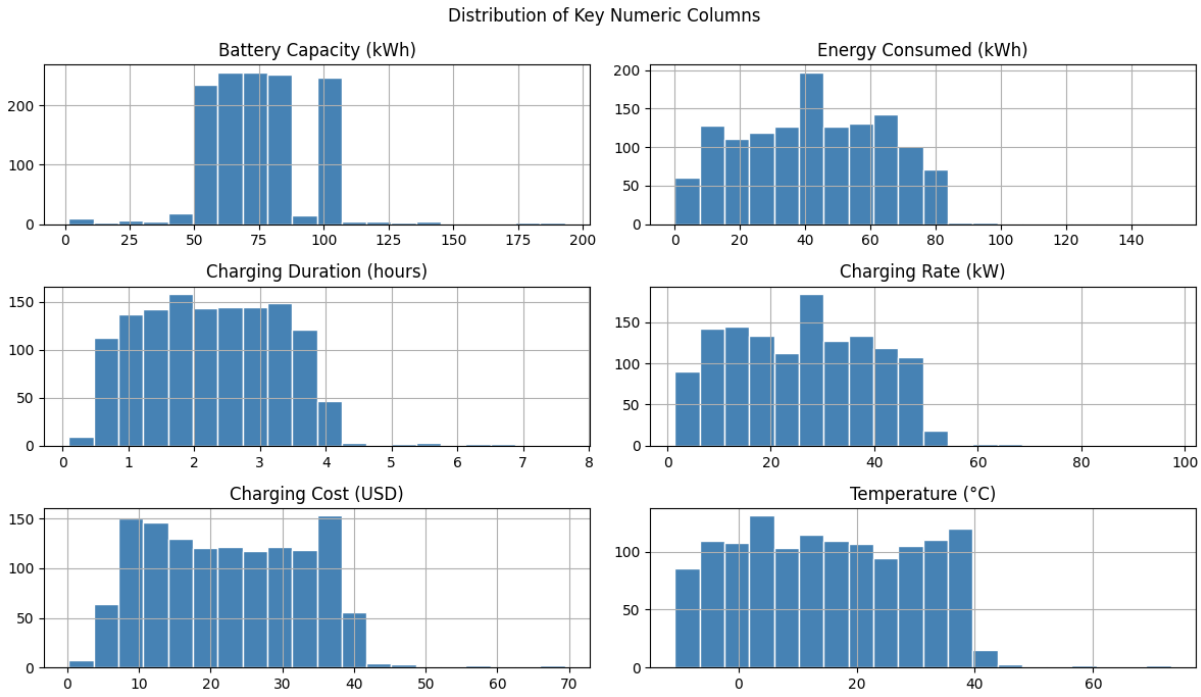


Figure 3.1: Histograms of key numeric columns

Most columns are spread fairly evenly across their range, without a single strong peak. Battery Capacity mostly takes a few fixed values (50, 62, 75, 85 and 100 kWh), which matches the idea of standard battery sizes. Energy Consumed and Charging Rate each show one taller bar; these sit at the column medians (42.7 kWh and 25.6 kW), where the gaps of the original file were filled.

3.10 Key Observations from EDA

- The file used has no missing values, no duplicate rows and no battery percentages above 100%, so all 1,320 rows were kept.
- Four columns (two IDs and two timestamps) were dropped and one new column, Is Weekend, was added.
- Categories such as vehicle model, city and charger type are almost evenly balanced.
- Numeric columns are spread evenly with no strong peaks, which is unusual for real-world data and is studied further in Objective 4.

4. Analysis on Dataset

This chapter presents the five analyses. For each analysis the following points are covered: (i) Introduction, (ii) General Description, (iii) Specific Requirements, functions and formulas, (iv) Analysis results and (v) Visualization.

4.1 Analysis 1: User and Vehicle Profile

i. Introduction

Before studying cost or time, we need to know who is charging. This analysis counts how many sessions belong to each vehicle model, user type and charger type. Station operators can use this to decide which chargers to install and which customers to focus on.

ii. General Description

Three categorical columns are studied: Vehicle Model (5 models), User Type (3 types) and Charger Type (3 types). For each one, the number of sessions in each category is counted and shown as a bar chart. A crosstab is also made to see which charger types each user type prefers.

iii. Specific Requirements, Functions and Formulas

- **value_counts()** – counts how many times each category appears.
- **value_counts(normalize=True) × 100** – gives the percentage share of each category.
- **pd.crosstab(A, B)** – builds a table that counts every combination of two categories.
- **plot(kind="bar")** – draws a bar chart from the counts.

$$\textit{Percentage share} = (\textit{Sessions in category} \div \textit{Total sessions}) \times 100$$

```
df["Vehicle Model"].value_counts()
(df["User Type"].value_counts(normalize=True) * 100).round(1)
pd.crosstab(df["User Type"], df["Charger Type"])
```

iv. Analysis Results

Table 4.1: Session counts by vehicle model, user type and charger type

Category	Value	Sessions	Share (%)
Vehicle Model	Tesla Model 3	280	21.2
Vehicle Model	Hyundai Kona	266	20.2
Vehicle Model	Nissan Leaf	260	19.7
Vehicle Model	BMW i3	258	19.5

Vehicle Model	Chevy Bolt	256	19.4
User Type	Commuter	476	36.1
User Type	Long-Distance Traveler	437	33.1
User Type	Casual Driver	407	30.8
Charger Type	Level 1	459	34.8
Charger Type	Level 2	431	32.7
Charger Type	DC Fast Charger	430	32.6

Table 4.2: Crosstab of user type and charger type

User Type	DC Fast Charger	Level 1	Level 2
Casual Driver	138	141	128
Commuter	142	160	174
Long-Distance Traveler	150	158	129

Tesla Model 3 is the most common vehicle (280 sessions) and **Chevy Bolt** the least common (256). **Commuters** form the largest user group (36.1%), followed by Long-Distance Travelers (33.1%) and Casual Drivers (30.8%). **Level 1** is the most used charger (459 sessions). The crosstab shows that Commuters use Level 2 chargers the most (174 sessions), which makes sense because Level 2 chargers are common at workplaces and homes.

Overall, every group is fairly balanced; the biggest and smallest groups differ by only a small number of sessions. No single vehicle, user type or charger dominates the data.

v. Visualization

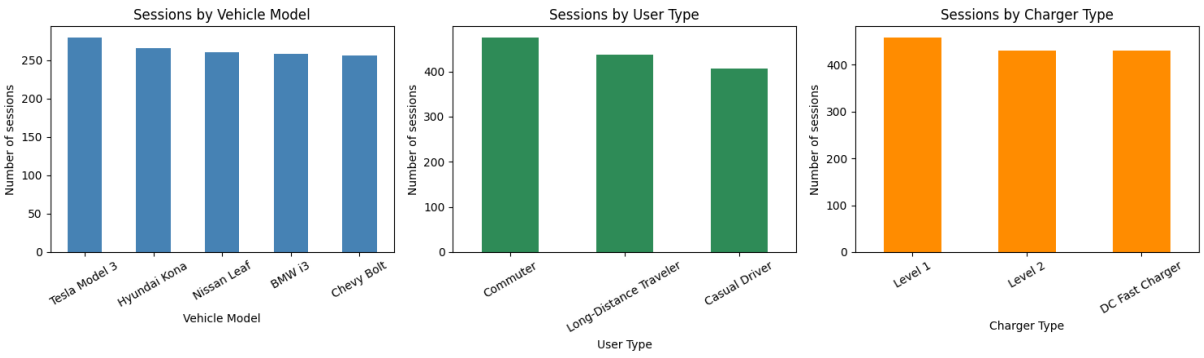


Figure 4.1: Number of sessions by vehicle model, user type and charger type

The three bar charts have nearly equal heights, which visually confirms that the categories are balanced.

4.2 Analysis 2: Charging Cost and Energy by Location and Charger Type

i. Introduction

Cost is one of the most important concerns for EV owners. This analysis checks whether charging is more expensive in some cities or on some types of charger, and whether the energy taken per session is different.

ii. General Description

The data is grouped by city (Charging Station Location) and by Charger Type. For each group the average charging cost, average energy consumed and average duration are calculated and compared using bar charts. A histogram of charging cost shows how costs are spread overall.

iii. Specific Requirements, Functions and Formulas

- **groupby(column)[...].mean()** – splits the rows into groups and calculates the average for each group.
- **sort_values(ascending=False)** – sorts results from highest to lowest.
- **plt.hist()** – draws a histogram; **plt.axvline()** draws the mean line on it.

$$Mean\ (average) = (x_1 + x_2 + \ldots + x_n) \div n$$

```
cost_by_city = df.groupby("Charging Station Location")["Charging Cost
(USD)"].mean()
charger_summary = df.groupby("Charger Type")[["Charging Cost (USD)",
                  "Energy Consumed (kWh)", "Charging Duration
(hours)"]].mean()
```

iv. Analysis Results

Table 4.3: Average cost and energy by city

City	Avg. cost (USD)	Avg. energy (kWh)
Chicago	23.72	43.46
Los Angeles	22.54	41.48
New York	22.50	42.25
Houston	22.09	44.06
San Francisco	21.99	42.19

Table 4.4: Average cost, energy and duration by charger type

Charger type	Avg. cost (USD)	Avg. energy (kWh)	Avg. duration (h)
DC Fast Charger	23.60	41.37	2.28

Level 1	21.66	41.66	2.24
Level 2	22.45	44.97	2.28

Chicago has the highest average cost (USD 23.72) and **San Francisco** the lowest (USD 21.99), a difference of only USD 1.72. **DC Fast Chargers** are the most expensive on average (USD 23.60) and **Level 1** chargers the cheapest (USD 21.66). This matches real life, where fast charging is usually priced higher.

An unexpected result is that DC Fast Charger sessions are **not shorter** than Level 1 sessions (2.28 h compared with 2.24 h). In reality, a fast charger should finish much sooner. This is the first sign that the dataset may not behave like real-world data.

The cost histogram shows that most sessions cost between about USD 5 and USD 40, spread fairly evenly, with a mean of USD 22.55. Only a few sessions cost more than USD 45.

v. Visualization

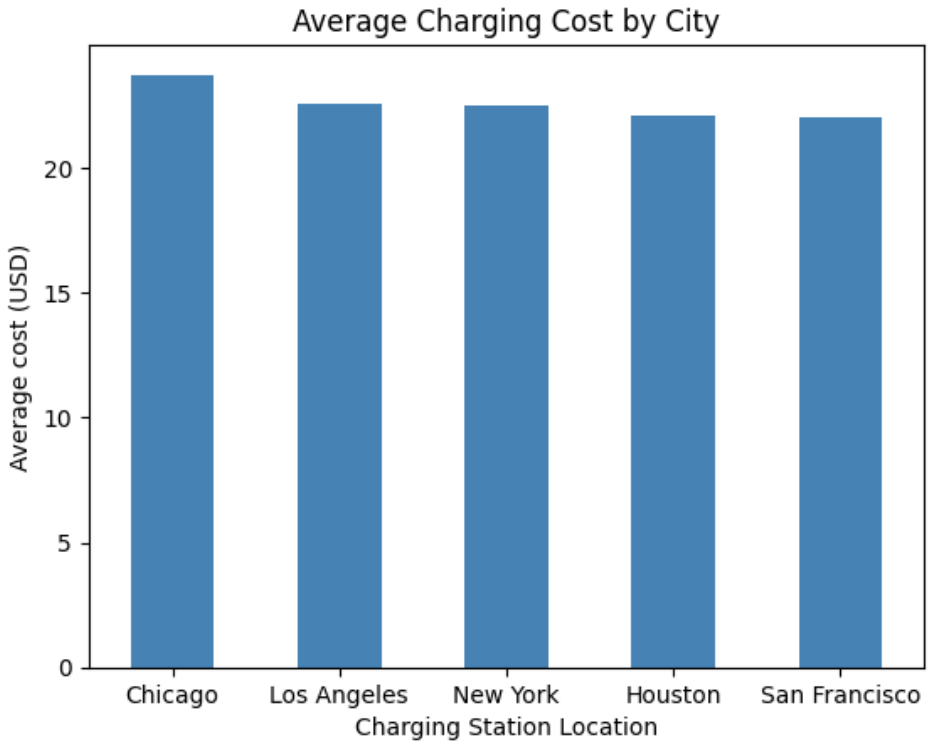


Figure 4.2: Average charging cost by city

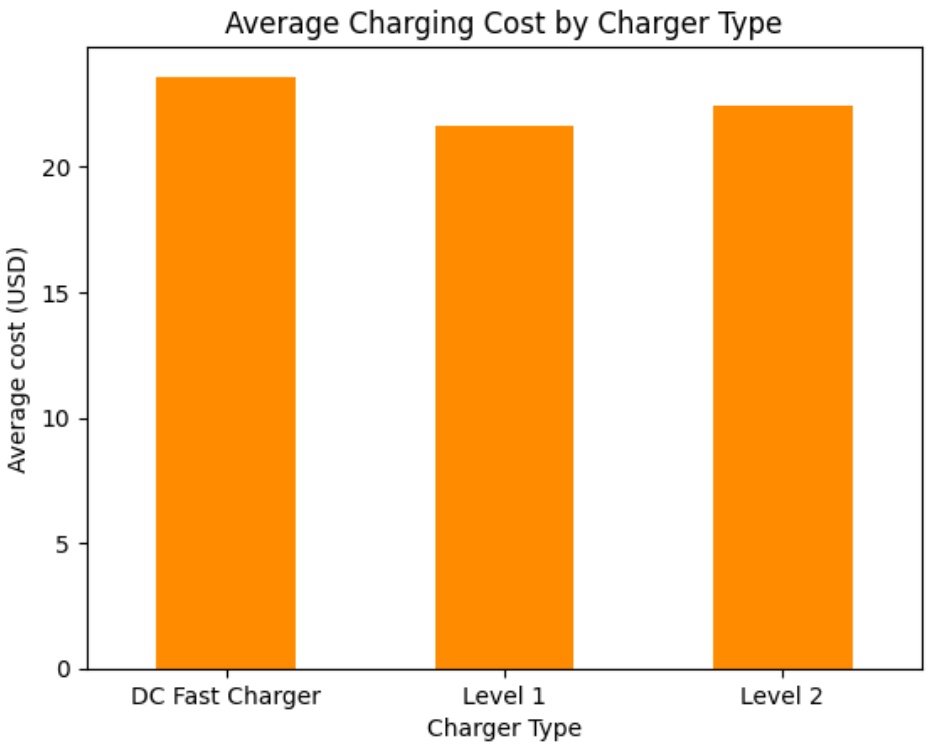


Figure 4.3: Average charging cost by charger type

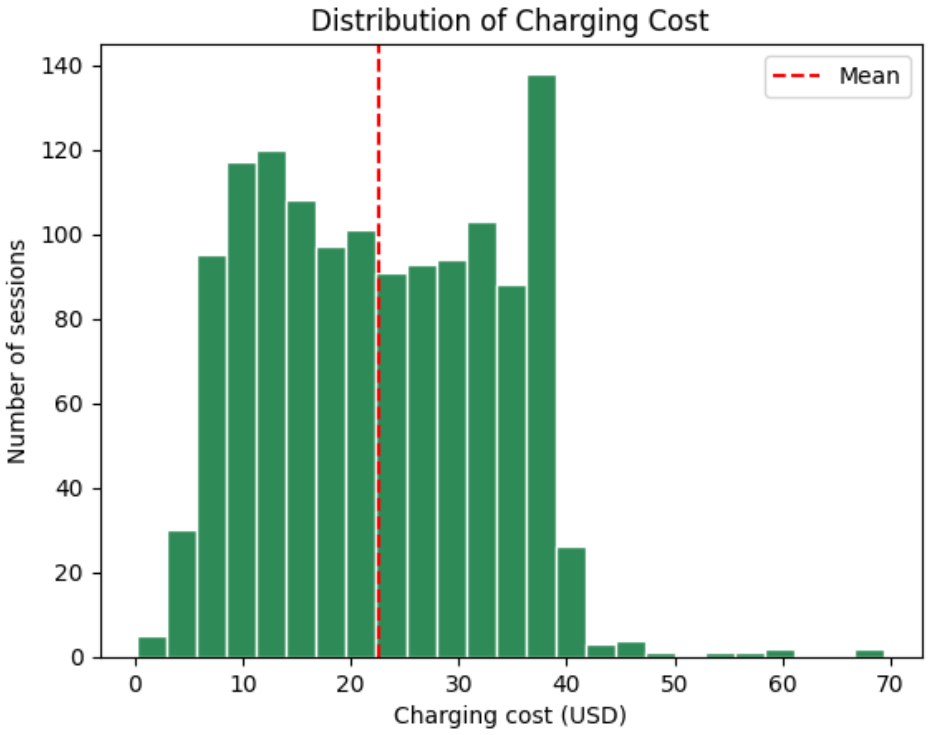


Figure 4.4: Distribution of charging cost (red line = mean)

4.3 Analysis 3: Time-based Charging Patterns

i. Introduction

Knowing *when* people charge helps operators plan for busy hours and helps electricity companies manage the load on the power grid. This analysis studies the time of day and the day of the week of charging sessions, and compares weekday and weekend sessions.

ii. General Description

Sessions are counted for each Time of Day (Morning, Afternoon, Evening, Night) and each Day of Week. The engineered column Is Weekend is used to compare average cost, energy and duration between weekdays and weekends.

iii. Specific Requirements, Functions and Formulas

- **value_counts().reindex(order)** – counts sessions and puts them in a natural order (Monday to Sunday) instead of from largest to smallest.
- **isin(["Saturday", "Sunday"])** – checks whether the day is a weekend day; used to create Is Weekend.
- **groupby("Is Weekend").mean()** – compares the averages of weekday and weekend sessions.

```
df["Time of
Day"].value_counts().reindex(["Morning","Afternoon","Evening","Night"])
df.groupby("Is Weekend")[["Charging Cost (USD)", "Energy Consumed
(kWh)",
                          "Charging Duration (hours)"]].mean()
```

iv. Analysis Results

Table 4.5: Sessions and average cost by time of day

Time of Day	Sessions	Share (%)	Avg. cost (USD)
Morning	336	25.5	22.41
Afternoon	310	23.5	23.14
Evening	362	27.4	22.38
Night	312	23.6	22.31

Table 4.6: Sessions by day of week

Day	Sessions
Monday	185
Tuesday	200

Wednesday	197
Thursday	154
Friday	188
Saturday	205
Sunday	191

Table 4.7: Weekday vs weekend comparison

	Avg. cost (USD)	Avg. energy (kWh)	Avg. duration (h)
Weekday	22.83	42.67	2.31
Weekend	21.89	42.59	2.16

Evening is the busiest time of day (362 sessions), which makes sense because many people charge after returning from work. **Afternoon** is the quietest (310 sessions). By day, **Saturday** has the most sessions (205) and **Thursday** has the fewest (154).

Weekend sessions are slightly cheaper (USD 21.89 vs USD 22.83) and slightly shorter (2.16 h vs 2.31 h). The differences are small, so charging behaviour is broadly similar across the week. Average cost by time of day is also almost the same, between USD 22.31 and USD 23.14.

v. Visualization

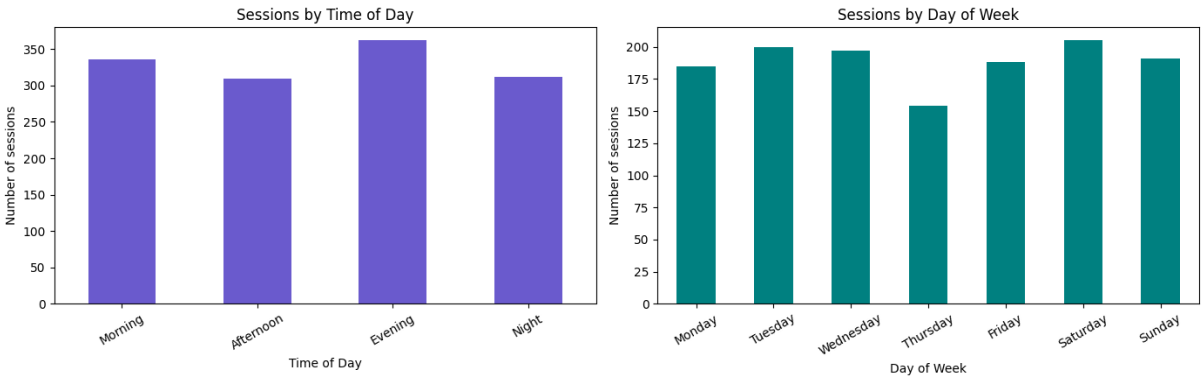


Figure 4.5: Number of sessions by time of day and by day of week

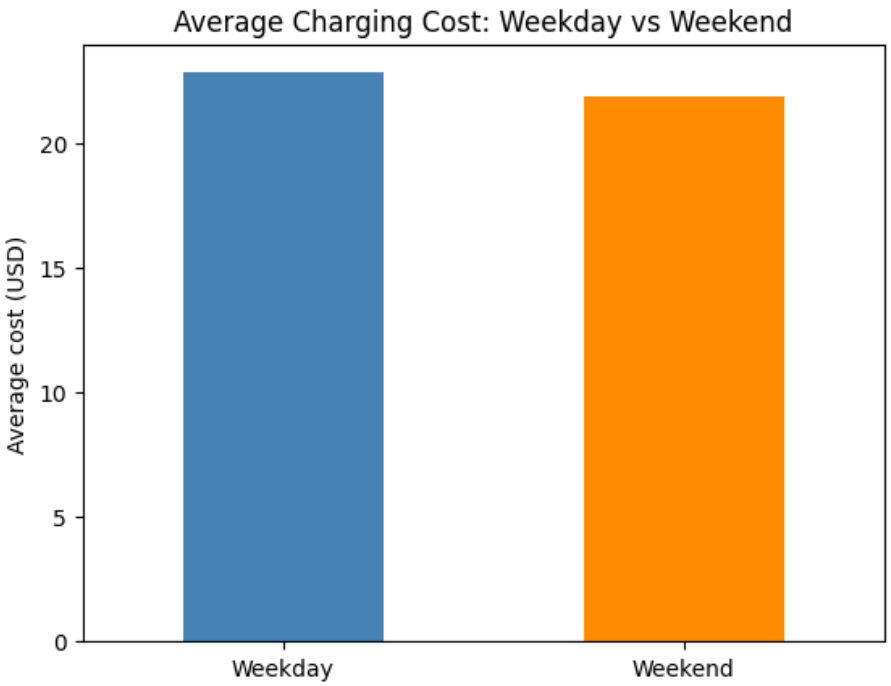


Figure 4.6: Average charging cost on weekdays vs weekends

4.4 Analysis 4: Relationships between Numeric Factors

i. Introduction

In real life we expect some columns to be related. For example, a session that delivers more energy should cost more, and a longer session should deliver more energy. This analysis measures these relationships using correlation.

ii. General Description

The Pearson correlation is calculated between every pair of the eleven numeric columns (the ten original numeric columns plus Is Weekend) and shown as a heatmap. The correlation of each column with Charging Cost is listed separately, because cost is the target for prediction in Analysis 5. Two scatter plots (Energy vs Cost and Duration vs Energy) show the relationships visually.

iii. Specific Requirements, Functions and Formulas

Correlation (r) is a number between −1 and +1:

- **r close to +1**: when one value increases, the other also increases.
- **r close to −1**: when one value increases, the other decreases.
- **r close to 0**: there is no straight-line relationship.

Commonly used guide: |r| below 0.3 is weak, 0.3 to 0.7 is moderate, and above 0.7 is strong.

$$r = \Sigma (x - \bar{x})(y - \bar{y}) \div \sqrt{[\Sigma (x - \bar{x})^2 \times \Sigma (y - \bar{y})^2]}$$

Here $\bar{x}$ and $\bar{y}$ are the means of the two columns. Functions used: **df.corr()** to calculate all correlations, **sns.heatmap()** to colour the correlation table, and **plt.scatter()** for scatter plots.

```
corr = df.select_dtypes(include="number").corr()
sns.heatmap(corr, annot=True, cmap="coolwarm")

corr["Charging Cost (USD)"].drop("Charging Cost
(USD)").sort_values(key=abs, ascending=False)
```

iv. Analysis Results

Table 4.8: Correlation of each numeric column with charging cost

Column	Correlation with Charging Cost	Strength
Temperature (°C)	+0.060	Very weak / none
State of Charge (End %)	-0.056	Very weak / none
Is Weekend	-0.040	Very weak / none
Charging Duration (hours)	+0.016	Very weak / none
State of Charge (Start %)	-0.010	Very weak / none
Distance Driven (since last charge) (km)	-0.008	Very weak / none
Energy Consumed (kWh)	-0.008	Very weak / none
Vehicle Age (years)	-0.006	Very weak / none
Charging Rate (kW)	-0.001	Very weak / none
Battery Capacity (kWh)	-0.000	Very weak / none

Every correlation in the heatmap lies between **-0.07 and 0.09**, which is extremely close to zero. The strongest relationship with cost is Temperature, at only +0.060. Even Energy Consumed and Charging Cost have a correlation of only -0.008, although in reality cost is calculated from the energy used. Similarly, Charging Duration and Energy Consumed have a correlation of just 0.028.

The scatter plots look like random clouds of points with no upward or downward trend. Together with the result from Analysis 2 (fast chargers not being faster), this strongly suggests that the dataset is **synthetic**, meaning its values were generated randomly and independently, rather than recorded from real charging stations. This is an important finding, because it tells us what to expect from the prediction models in the next analysis.

v. Visualization

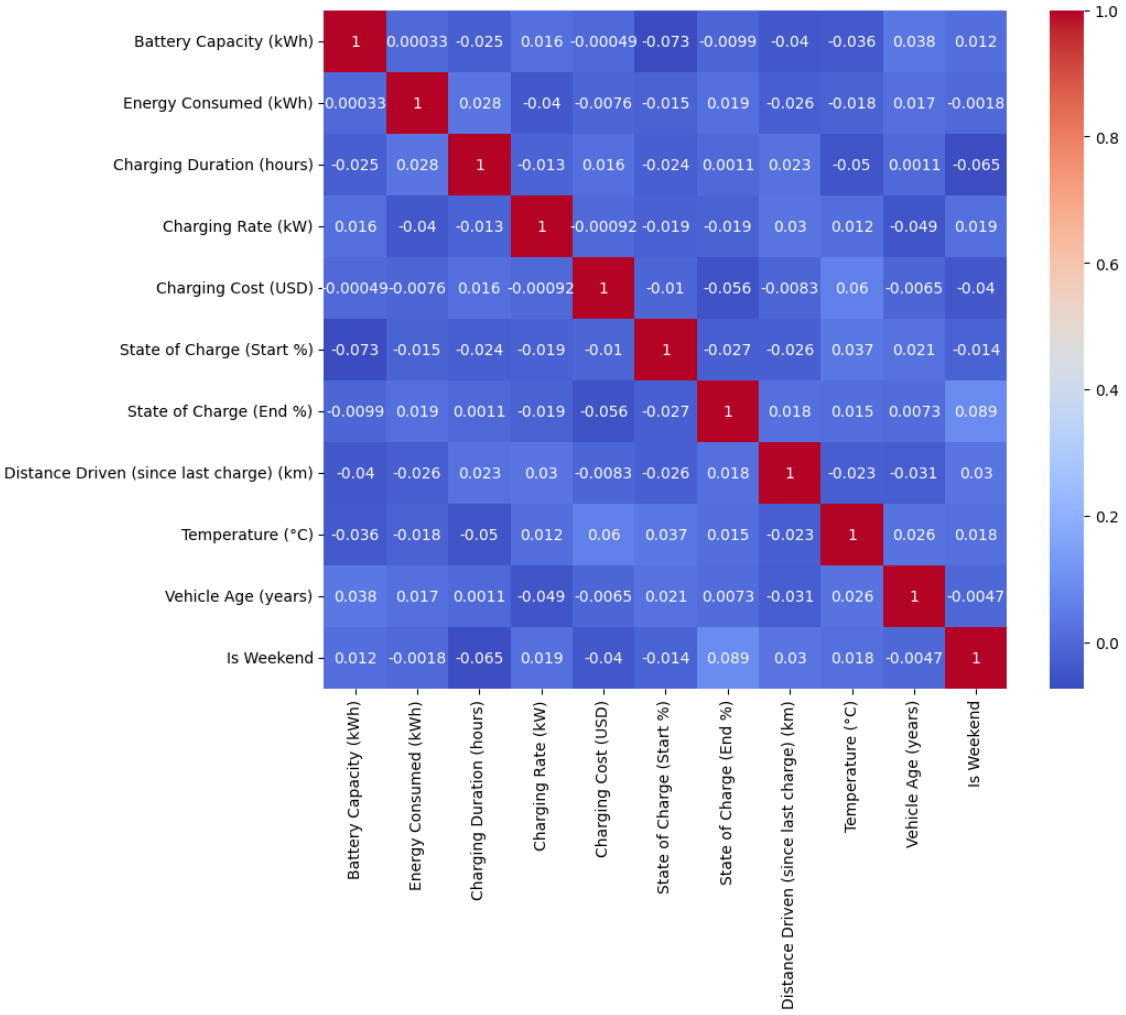


Figure 4.7: Correlation heatmap of numeric columns

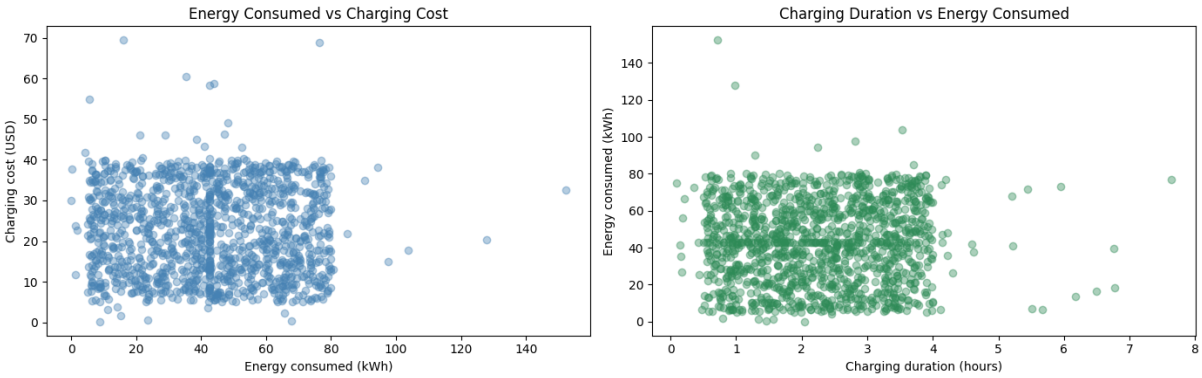


Figure 4.8: Scatter plots: energy vs cost, and duration vs energy

4.5 Analysis 5: Predicting Charging Cost with Machine Learning

i. Introduction

If the cost of a charging session could be predicted before it starts, users could plan their charging and operators could set fair prices. This analysis tests whether four simple machine learning models can predict Charging Cost (USD) from the other columns. Because the value to predict is a number, this is a **regression** problem.

ii. General Description

The steps followed are:

1. **Features and target:** y = Charging Cost; X = all other columns.
2. **One-hot encoding:** text columns (Vehicle Model, City, Time of Day, Day of Week, Charger Type, User Type) are converted into 0/1 columns using pd.get_dummies(drop_first=True). This gives **31 input features**. Example: Charger Type becomes two columns, Level 1 and Level 2; if both are 0, the charger is a DC Fast Charger.
3. **Train/test split:** 80% of rows (1056) are used for training and 20% (264) for testing, with random_state=42 so the result is the same every time.
4. **Scaling:** StandardScaler is applied for KNN, because KNN uses distances and columns with large numbers would otherwise dominate. The scaler is fitted on training data only, so no information from the test data leaks into training.
5. **Baseline:** a simple guess that always predicts the average training cost. A useful model must perform better than this.
6. **Train and evaluate** the four models and compare their errors.

The four models and why each was chosen:

Table 4.9: The four regression models used

Model	How it works (simple idea)	Why it is used	Settings
Linear Regression	Fits a straight line (plane): cost = $b_0 + b_1x_1 + b_2x_2 + \ldots$	Simplest regression model; easy to explain; standard starting point	Default
Decision Tree	Asks yes/no questions (e.g. Energy > 40?) and predicts the average cost of each final group	Can capture non-linear patterns; easy to visualise	max_depth=5 (to avoid overfitting)
K-Nearest Neighbors	Finds the 5 most similar sessions in the training data and averages their cost	Very intuitive: similar sessions should cost similar amounts	n_neighbors=5, scaled data

Random Forest	Builds 100 decision trees on random samples and averages their predictions	Usually more accurate and stable than one tree; gives feature importance	n_estimators=100

iii. Specific Requirements, Functions and Formulas

Scikit-learn functions used: **train_test_split**, **StandardScaler**, **LinearRegression**, **DecisionTreeRegressor**, **KNeighborsRegressor**, **RandomForestRegressor**, and the metrics **mean_absolute_error**, **mean_squared_error** and **r2_score**. Every model follows the same two steps: **fit()** to learn from training data and **predict()** to make predictions for test data.

```
X = pd.get_dummies(df.drop(columns=["Charging Cost (USD)"]),
drop_first=True, dtype=int)
y = df["Charging Cost (USD)"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)          # learn
rf_pred = rf.predict(X_test)      # predict
```

The models are evaluated with three metrics, where y_i is the actual cost, $\hat{y}_i$ the predicted cost, $\bar{y}$ the mean actual cost and n the number of test rows:

$$MAE = (1/n) \times \Sigma | y_i - \hat{y}_i |$$

$$RMSE = \sqrt{[(1/n) \times \Sigma (y_i - \hat{y}_i)^2]}$$

$$R^2 = 1 - \Sigma (y_i - \hat{y}_i)^2 \div \Sigma (y_i - \bar{y})^2$$

Table 4.10: Evaluation metrics

Metric	Meaning in simple words	Better when
MAE	Average size of the error, in dollars	Lower
RMSE	Like MAE, but big errors are punished more	Lower
R^2	Share of the variation in cost explained by the model. 1 = perfect, 0 = same as guessing the average, negative = worse than the average	Higher

iv. Analysis Results

Table 4.11: Performance of the four models and the baseline on test data

Model	MAE (USD)	RMSE (USD)	R^2 score
Baseline (average)	9.44	11.20	-0.000
Linear Regression	9.39	11.16	0.007
Decision Tree	9.55	11.38	-0.033
KNN (k=5)	10.00	11.94	-0.138

Random Forest	9.60	11.37	-0.031

The baseline, which always predicts the average cost, has an MAE of **USD 9.44** and an R^2 of 0.000. **Linear Regression** is the best of the four models, with MAE **USD 9.39** and R^2 **0.007**. It is the only model that is better than the baseline, but only by USD 0.05 (about 5 cents) per session, and an R^2 of 0.007 means it explains less than 1% of the variation in cost. The other three models are slightly **worse** than the baseline, and **KNN (k=5)** is the worst (MAE USD 10.00, R^2 -0.138).

What this means:

- In practice, **no model gives a useful improvement** over simply guessing the average cost. This agrees with Analysis 4, where Charging Cost had almost zero correlation with every other column. A model cannot learn a relationship that is not present in the data.
- **Linear Regression came out on top** because, when there is no real pattern, a straight-line model ends up predicting values very close to the average and does not chase random noise. Its tiny improvement (USD 0.05) is too small to be meaningful and could change with a different random train/test split.
- **KNN performed worst** (R^2 = -0.138). Its prediction is the average of only 5 neighbours, and when the columns are unrelated to cost these 5 neighbours are effectively random, so its predictions jump around more than the overall average does.
- **Decision Tree and Random Forest** were slightly worse than the baseline because trees try to find splits in the training data; with no real pattern, these splits only capture noise.
- **Random Forest** gives feature importances (Figure 4.12), with Temperature, State of Charge (Start %) and Energy Consumed at the top. But since the model is not better than the baseline, these importances only show which columns the trees happened to split on; they are **not** real causes of cost in this dataset.
- In the actual vs predicted plot (Figure 4.11), the Random Forest predictions stay in a narrow band of about USD 17–29, while actual costs range from about USD 5 to 55 (one session is near USD 69). A good model would place the points along the red diagonal line.

Key learning: a model should always be compared with a simple baseline. Looking only at an MAE of about USD 9.4–10 might suggest the models are reasonable, but the baseline shows that almost the same error is achieved without any machine learning at all.

v. Visualization

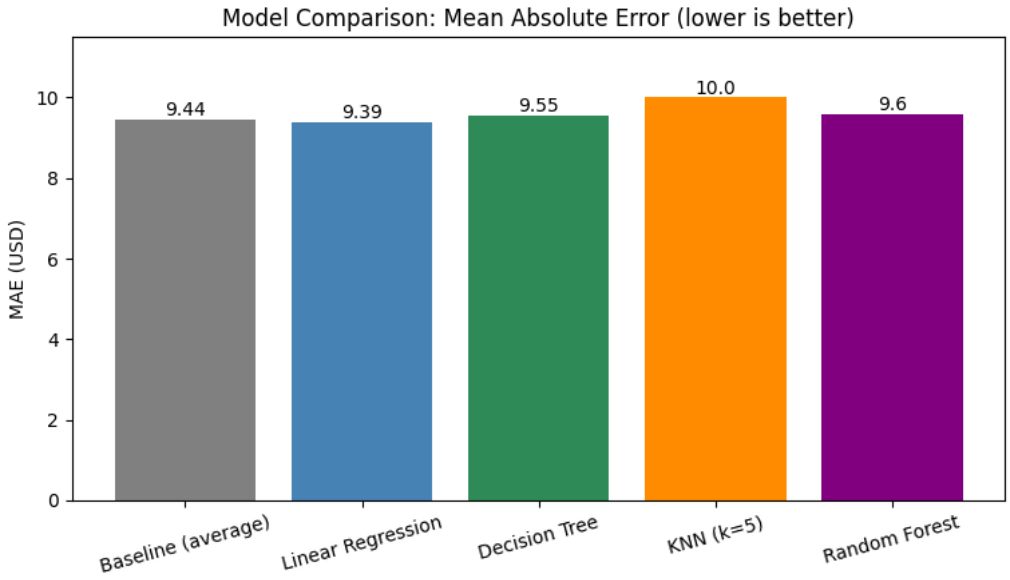


Figure 4.9: Model comparison by Mean Absolute Error (lower is better)

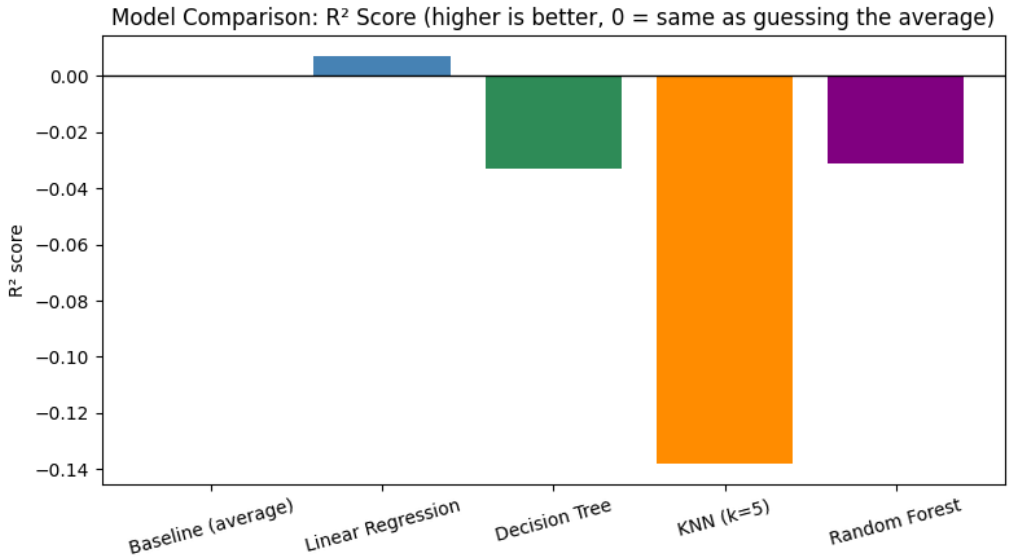


Figure 4.10: Model comparison by R^2 score (higher is better)

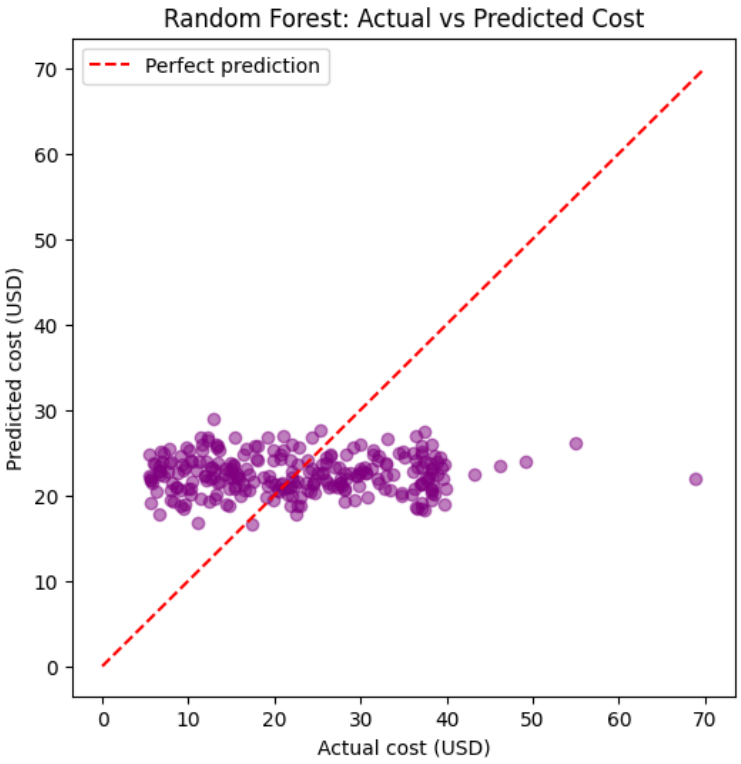


Figure 4.11: Random Forest: actual vs predicted charging cost

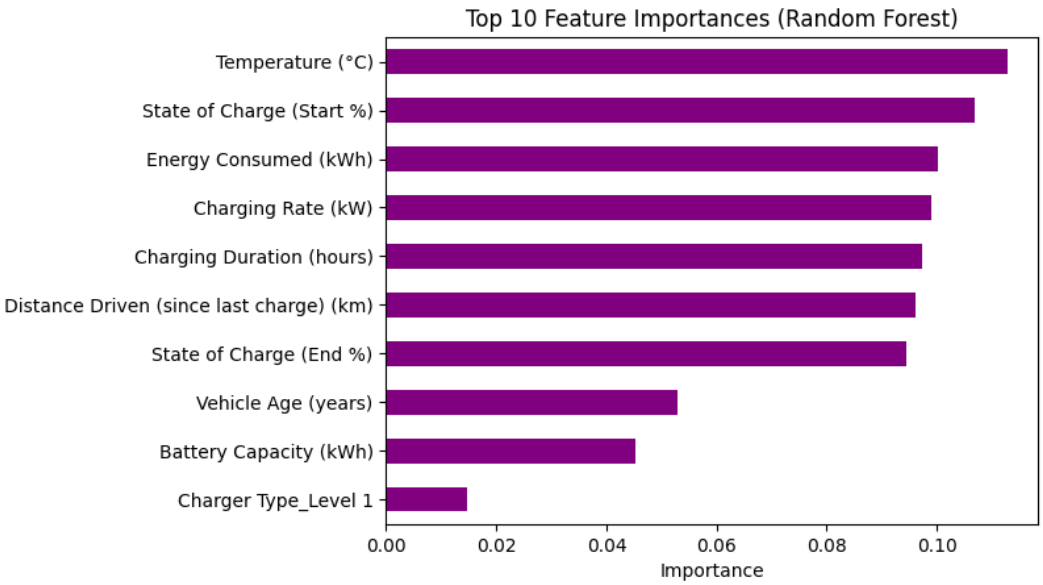


Figure 4.12: Top 10 feature importances from the Random Forest

5. Conclusion

This project analysed 1,320 electric vehicle charging sessions using basic Python data science tools. The data was checked for missing values, duplicates and impossible battery percentages; the file used had none of these problems, so all 1,320 sessions were used for analysis after dropping four unneeded columns and adding an Is Weekend column. The main conclusions for each objective are:

1. **User and vehicle profile:** The data is well balanced. Tesla Model 3 is the most common vehicle, Commuters are the largest user group (36.1%) and Level 1 is the most used charger, but all by small margins.
2. **Cost by location and charger type:** Chicago is the most expensive city and DC Fast Chargers the most expensive charger type, but differences are small (around USD 2). DC Fast Charger sessions were not shorter than others, which is unrealistic.
3. **Time patterns:** Evening is the busiest time to charge and Saturday the busiest day. Weekend sessions are slightly cheaper, but overall behaviour is similar across the week.
4. **Relationships:** All numeric columns have near-zero correlation (between -0.07 and 0.09). Even energy and cost are unrelated, which strongly suggests that the dataset is synthetic.
5. **Prediction:** Linear Regression, Decision Tree, KNN and Random Forest were trained and compared. Linear Regression was best (MAE USD 9.39, R^2 0.007) but beat the simple average baseline by only about 5 cents, and KNN performed worst. No model is practically useful for predicting cost in this dataset, which is explained by Analysis 4.

Overall, the project demonstrates the complete data science workflow: loading, understanding, cleaning, feature engineering, exploratory analysis, visualization, model training and evaluation. It also shows two important lessons: (1) always check the data for errors before using it, and (2) always compare a model against a simple baseline before trusting its results.

6. Future Scope

- **Use real-world data:** Repeat the same analysis on real charging data (for example from a city open-data portal or a charging network), where cost is linked to energy and tariffs. The same four models are expected to perform much better there.
- **Add useful features:** Include electricity price per kWh, station operator, peak/off-peak tariff and session start hour. Price per kWh × energy would make cost much easier to predict.
- **Fix time columns:** Make sure Day of Week and Time of Day match the actual timestamps, so time-based analysis is reliable.
- **Try other targets:** Predict energy consumed or charging duration, or classify the charger type, using the same simple models.
- **Model tuning and cross-validation:** Try different values of k for KNN and max_depth for the Decision Tree, and use k-fold cross-validation for more reliable scores.
- **Simple dashboard:** Present the findings in an interactive dashboard (for example using Power BI or a simple web app) for station operators.

7. References

1. Valakhorasani, "Electric Vehicle Charging Patterns," Kaggle, Oct. 2024. [Online]. Available: https://www.kaggle.com/datasets/valakhorasani/electric-vehicle-charging-patterns
2. W. McKinney, Python for Data Analysis, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022.
3. J. VanderPlas, Python Data Science Handbook. Sebastopol, CA: O'Reilly Media, 2016.
4. A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022.
5. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
6. Pandas Documentation. [Online]. Available: https://pandas.pydata.org/docs/
7. Scikit-learn User Guide. [Online]. Available: https://scikit-learn.org/stable/user_guide.html
8. Matplotlib Documentation. [Online]. Available: https://matplotlib.org/stable/
9. Seaborn Documentation. [Online]. Available: https://seaborn.pydata.org/
10. NumPy Documentation. [Online]. Available: https://numpy.org/doc/